

Manuale Utente - CineMax

- Bin Alessio (Matricola: 762387) - VA
- Modena Matteo (Matricola: 765099) - VA
- Baroncelli Luca (Matricola: 761582) - VA

Università degli Studi dell'Insubria

Laboratorio Interdisciplinare A - 2026

Data: Maggio 2026

Versione Documento: 1.0

Indice

1. Introduzione
2. Architettura del Sistema
 - 2.1 Pattern Architetturali
 - 2.2 Struttura dei Package
3. Modello Dati
 - 3.1 Gerarchia delle Classi Mode
 - 3.2 Persistenza Dati
4. Implementazione State Machine
 - 4.1 Pattern State con Enum
 - 4.2 Gestione dello Stack
 - 4.3 Transizioni di Stato
5. Layer Controller
 - 5.1 FilmController
 - 5.2 AutenticazioniController
 - 5.3 Pattern di Gestione Errori
6. Layer Utility
 - 6.1 GestoreProiezione
 - 6.2 GestoreUtenti
 - 6.3 [Sistema di Cifratura](#63-sistema-di-cifratura)
7. Interfaccia Testuale (TUI)
 - 7.1 Rendering con ANSI Escape Sequences
 - 7.2 Gestione Input Dinamico
 - 7.3 Sistema di Paginazione
8. Scelte Algoritmiche
 - 8.1 Algoritmo di Ricerca con Stream API
 - 8.2 Gestione Paginazione
 - 8.3 Validazione Input
9. Gestione Eccezioni
10. JavaDoc e Documentazione Codice
11. Considerazioni su Performance e Scalabilità
12. Bibliografia Tecnica

1. Introduzione

CineMax è un sistema di gestione cinematografica basato su interfaccia testuale (TUI) sviluppato in linguaggio Java. L'applicazione permette di gestire proiezioni cinematografiche, prenotazioni di biglietti e utenti per cinema monosala da 200 posti attraverso un'interfaccia grafica testuale intuitiva.

Il sistema supporta quattro tipologie di utenti:

- Cliente Ospite: può navigare e cercare film senza registrazione
- Cliente Registrato: può effettuare prenotazioni e gestire tali prenotazioni
- Proiezionista: può inserire, modificare ed eliminare proiezioni
- Bigliettaio: può visualizzare la programmazione, cercare prenotazioni e gestire le prenotazioni

L'applicazione utilizza un'architettura State Machine basata su uno Stack LIFO (Last-In-First-Out) per la navigazione tra le schermate. Ogni azione dell'utente può effettuare un push (avanzamento), un pop (ritorno) o un clear (reset/logout) dello stato applicativo.

2. Architettura del Sistema

2.1 Pattern Architetturali

Pattern State Machine

L'applicazione implementa una State Machine basata su Stack LIFO per la gestione della navigazione. Ogni stato rappresenta una schermata dell'applicazione e contiene la logica associata.

```
```java
// Stack globale per gestire gli stati
public static Stack<StatoMenu> stackRecord = new Stack<>();

// Enum che rappresenta gli stati
public enum StatoMenu {
 BENVENUTO, LOGIN, REGISTRAZIONE,
 MENU_CLIENTI, MENU_PROEZIONISTA, MENU_BIGLIETTAIO,
 CERCA_FILM, VISUALIZZA_PROGRAMMAZIONE,
 // ... altri stati
}
```
```

Vantaggi dell'approccio:

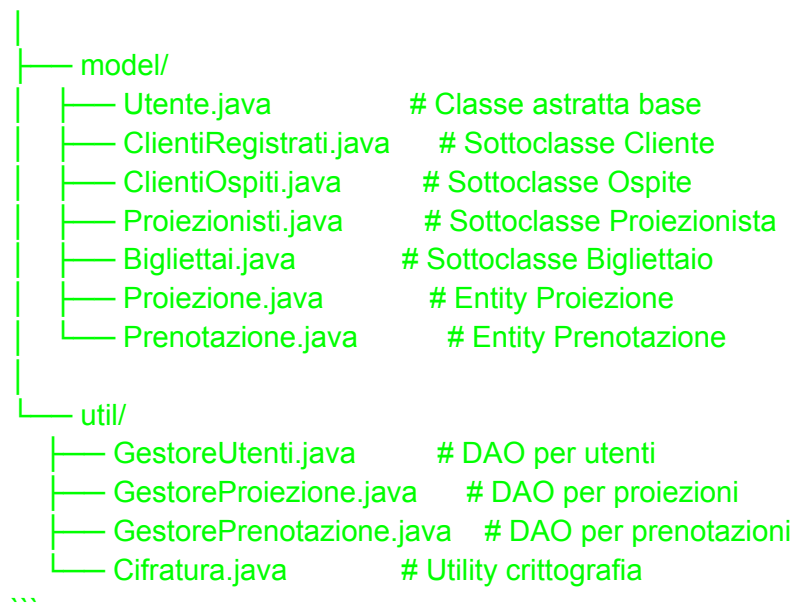
- Navigazione backward naturale (pop dallo stack)
- Stato centralizzato e tracciabile
- Facile debugging del flusso applicativo

Pattern MVC (Model-View-Controller)

Questo pattern permette di dividere le responsabilità all'interno dell'applicazione, dividendo il codice in 3 parti fondamentali. Il modello che si occupa di gestire dati, regole di business, regole di calcolo e l'interazione con il flat-file system. La vista che rappresenta l'interfaccia grafica con cui l'utente interagisce. Il controllore che coordina il modello e la vista

2.2 Struttura dei Package

```
```
cinemax/
├── CineMax.java # Entry point, gestisce ciclo vita
├── MenuMangaer.java # Enum Stati + logica associata
├── LogicaStatiManager.java # Transizioni stato e form
├── CinemaxTUI.java # Rendering interfaccia
├── CostantiForm.java # Enum per indici array form
├──
├── controller/
│ ├── FilmController.java # Logica business film/proiezioni
│ └── AutenticazioniController.java # Logica autenticazione
└──
```
```



3. Modello Dati

3.1 Gerarchia delle Classi Model

Classe Astratta Utente

screenshot classe

Gerarchia di ereditarietà: La classe astratta Utente è ereditata dalle classi Clienti, Proiezionisti, Bigliettai; Clienti a sua volta è ereditata da ClientiRegistrati e ClientiOspiti. L'uso di una classe astratta permette il polimorfismo e la condivisione di attributi comuni, mantenendo flessibilità per specializzazioni future.

3.2 Persistenza Dati

Pattern DAO (Data Access Object)

Le classi `GestoreUtenti` e `GestoreProiezione` implementano il pattern DAO per separare la logica di accesso ai dati dal business logic.

```
```java
public class GestoreProiezione {
 private static List<Proiezione> listaProiezioni = new ArrayList<>();
 private static final String FILE_PATH = "data/proiezioni.csv";

 // CRUD Operations
 public static void inserisci(Proiezione p);
 public static void elimina(Proiezione p);
 public static void leggiProiezioni();
 public static void salvaSuFile();
 public static List<Proiezione> cercaProiezione(...);
}
```
```

I dati vengono caricati in strutture in-memory (`ArrayList`, `LinkedList`) all'avvio e sincronizzati su file dopo ogni modifica.

4. Implementazione State Machine

4.1 Pattern State con Enum

Definizione Stati

```
```java
public enum StatoMenu {
 BENVENUTO(
 new String[]{"accedi", "entra come guest", "registrati", "esci"},
 true, // visualizzaNumeri
 "cinemax", // nomeLogo
 "centro" // posizione
) {
 @Override
 public StatoMenu[] prossimi() {
 return new StatoMenu[]{LOGIN, MENU_GUEST, REGISTRAZIONE};
 }

 @Override
 public void eseguiLogicaAssociata() {
 String scelta = input.nextLine();
 if (scelta.equals("4")) System.exit(0);
 LogicaStatiManager.statoMenuSuccessivo(scelta);
 }
 },

 LOGIN(
 new String[]{"username", "password"},
 false,
 "accesso",
 "centro"
) {
 @Override
 public StatoMenu[] prossimi() {
 return new StatoMenu[]{MENU_CLIENTI, MENU_PROEZIONISTA,
 MENU_BIGLIETTAIO, BENVENUTO};
 }

 @Override
 public void eseguiLogicaAssociata() {
 String[] datiFormTmp = new String[getOpzioni().length];
 if (!LogicaStatiManager.prendiDatiForm(datiFormTmp, getOpzioni()))
 return;
 AutenticazioniController.gestisciLogin(datiFormTmp);
 }
 },
}
```

```

// ... altri stati
;

// Campi enum
private final String[] opzioni;
private final boolean visualizzaNumeri;
private final String nomeLogo;
private final String posizione;

// Costruttore
StatoMenu(String[] opzioni, boolean visualizzaNumeri,
 String nomeLogo, String posizione) {
 this.opzioni = opzioni;
 this.visualizzaNumeri = visualizzaNumeri;
 this.nomeLogo = nomeLogo;
 this.posizione = posizione;
}

// Metodi astratti
public abstract StatoMenu[] prossimi();
public abstract void eseguiLogicaAssociata();
}
...

```

Vantaggi del pattern:

1. Type-safety: Ogni stato è definito staticamente, errori rilevati a compile-time
2. Comportamento incapsulato: Logica specifica dello stato contenuta nel metodo `eseguiLogicaAssociata()`
3. Transizioni dichiarative: Array `prossimi()` definisce stati raggiungibili

## 4.2 Gestione dello Stack

Main Loop

```

```java
public class CineMax {
    public static Stack<StatoMenu> stackRecord = new Stack<>();
    public static String ruolo;

    public static void main(String[] args) {
        GestoreUtenti.caricaUtenti();
        stackRecord.push(StatoMenu.BENVENUTO);

        boolean running = true;
        ruolo = "cliente ospite";

        while (running) {
            if (stackRecord.isEmpty()) {

```

```

        System.out.println("Chiusura sistema...");
        break;
    }

    StatoMenu statoAttuale = stackRecord.peek();
    CinemaxTUI.renderizzaMenu(statoAttuale);
    statoAttuale.eseguiLogicaAssociata();
}
}
}
...

```

Operazioni stack:

- push(): Avanza a nuovo stato
- pop(): Torna allo stato precedente
- peek(): Legge stato corrente senza rimuoverlo
- clear(): Reset completo (logout)

4.3 Transizioni di Stato

Logica di Transizione

```

```java
public static void statoMenuSuccessivo(String indice) {
 try {
 int scelta = Integer.parseInt(indice);
 StatoMenu statoAttuale = CineMax.stackRecord.peek();
 StatoMenu[] possibiliStatiSucc = statoAttuale.prossimi();
 StatoMenu statoSuccessivo = possibiliStatiSucc[sceita - 1];

 if (statoSuccessivo == null) {
 CineMax.stackRecord.pop(); // Indietro
 return;
 }

 if (statoSuccessivo == StatoMenu.BENVENUTO) {
 CineMax.stackRecord.clear(); // Logout
 CineMax.ruolo = "cliente ospite";
 }

 CineMax.stackRecord.push(statoSuccessivo);

 } catch (NumberFormatException | ArrayIndexOutOfBoundsException e) {
 CineMax.stackRecord.push(StatoMenu.STATO_ERRORE);
 }
}
...

```

Gestione errori integrata: Input non validi portano automaticamente a `STATO\_ERRORE`, che mostra il messaggio e permette il ritorno.

## 5. Layer Controller

### 5.1 FilmController

Gestisce la logica business relativa a ricerca, visualizzazione e gestione proiezioni.

Sistema di Paginazione

```
```java
public class FilmController {
    public static List<Proiezione> filmPaginaTmp = new ArrayList<>();
    public static List<Proiezione> proiezioniTrovate = new ArrayList<>();
    public static Proiezione filmSelezionatoTmp;

    public static boolean esistenzaPaginaSuccessiva;
    public static int paginaCorrente = 0;
    public static final int ELEMENTI_PAGINA = 10;

    public static void aggiornaProezioniPerPagina() {
        filmPaginaTmp.clear();
        esistenzaPaginaSuccessiva = false;

        int indiceInizio = ELEMENTI_PAGINA * paginaCorrente;
        int indiceFine = indiceInizio + ELEMENTI_PAGINA;

        for (int i = indiceInizio;
            i < indiceFine && i < proiezioniTrovate.size();
            i++) {
            filmPaginaTmp.add(proiezioniTrovate.get(i));
        }

        if (indiceInizio + ELEMENTI_PAGINA < proiezioniTrovate.size())
            esistenzaPaginaSuccessiva = true;
        if (paginaCorrente > 0)
            esistenzaPrecedente = true;
    }
}
```
```

Pattern utilizzato: Pagination Pattern per gestire grandi dataset senza sovraccaricare la UI.

Inserimento Proiezione con Parsing

```
```java
public static void gestisciInserimentoProiezione(String[] datiFormTmp) {
    try {
        String titolo = datiFormTmp[Campi.ADD_TITOLO.i];
        String genere = datiFormTmp[Campi.ADD_GENERE.i];
    }
}
```

```

// ... altri campi
String giorno = datiFormTmp[Campi.ADD_GIORNO.i];
String mese = datiFormTmp[Campi.ADD_MESE.i];
String anno = datiFormTmp[Campi.ADD_ANNO.i];
String orario = datiFormTmp[Campi.ADD_ORA.i];

LocalDate data = LocalDate.of(
    Integer.parseInt(anno),
    Integer.parseInt(mese),
    Integer.parseInt(giorno)
);

Proiezione nuovaProiezione = new Proiezione(
    data, orario, titolo, genere, regista,
    Integer.parseInt(annoProduzione),
    Integer.parseInt(durata),
    Integer.parseInt(etaMinima),
    Double.parseDouble(costo.replace(",", ".")),
    Integer.parseInt(posti)
);

GestoreProiezione.inserisci(nuovaProiezione);
CineMax.stackRecord.pop();

} catch (IllegalArgumentException e) {
    LogicaStatiManager.messaggioErroreCorrente =
        "Errore durante l'aggiunta di una proiezione";
    CineMax.stackRecord.push(StatoMenu.STATO_ERRORE);
}
}
...

```

Pattern Enum per indici: `Campi.ADD\_TITOLO.i` fornisce accesso type-safe agli indici dell'array form.

5.2 AutenticazioniController

Gestione Login

```

```java
public static void gestisciLogin(String[] datiFormTmp) {
 String username = datiFormTmp[Campi.LOGIN_USER.i];
 String password = datiFormTmp[Campi.LOGIN_PASSWORD.i];

 try {
 Utente u = GestoreUtenti.login(username, password);

 switch (u.getRuolo()) {

```

```

 case "proiezionista":
 CineMax.stackRecord.push(StatoMenu.MENU_PROEZIONISTA);
 break;
 case "cliente registrato":
 CineMax.stackRecord.push(StatoMenu.MENU_CLIENTI);
 break;
 case "bigliettaio":
 CineMax.stackRecord.push(StatoMenu.MENU_BIGLIETTAIO);
 break;
 }

 CineMax.ruolo = u.getRuolo();

} catch (Exception e) {
 LogicaStatiManager.messaggioErroreCorrente =
 "credenziali non valide o utente inesistente";
 CineMax.stackRecord.push(StatoMenu.STATO_ERRORE);
}
}
...

```

#### #### Gestione Registrazione

```

```java
public static void gestisciRegistrazione(String[] datiFormTmp) {
    try {
        String nome = datiFormTmp[Campi.REG_NOME.i];
        String cognome = datiFormTmp[Campi.REG_COGNOME.i];
        String username = datiFormTmp[Campi.REG_USERNAME.i];
        String password = datiFormTmp[Campi.REG_PASSWORD.i];
        String confermaPassword = datiFormTmp[Campi.REG_CONFERMA_PASSWORD.i];
        String dataNascita = datiFormTmp[Campi.REG_DATA.i];
        String domicilio = datiFormTmp[Campi.REG_DOMICILIO.i];

        GestoreUtenti.registraUtente(
            nome, cognome, username, password,
            confermaPassword, dataNascita, domicilio
        );

        CineMax.stackRecord.clear();
        CineMax.stackRecord.push(StatoMenu.BENVENUTO);
        CineMax.ruolo = "cliente registrato";
        CineMax.stackRecord.push(StatoMenu.MENU_CLIENTI);

    } catch (IllegalArgumentException e) {
        LogicaStatiManager.messaggioErroreCorrente = e.getMessage();
        CineMax.stackRecord.push(StatoMenu.STATO_ERRORE);
    }
}

```

```
}  
...
```

5.3 Pattern di Gestione Errori

```
```java  
try {
 // Logica business
} catch (IllegalArgumentException e) {
 // Errore validazione dati
 LogicaStatiManager.messaggioErroreCorrente = e.getMessage();
 CineMax.stackRecord.push(StatoMenu.STATO_ERRORE);
} catch (Exception e) {
 // Errore generico
 LogicaStatiManager.messaggioErroreCorrente =
 "Errore imprevisto durante l'operazione";
 CineMax.stackRecord.push(StatoMenu.STATO_ERRORE);
}
...
```

**\*\*Variabile globale per messaggi\*\*:** `LogicaStatiManager.messaggioErroreCorrente` viene letta da `STATO\_ERRORE` per mostrare il messaggio appropriato.

---

## 6. Layer Utility

### 6.1 GestoreProiezione

Algoritmo di Ricerca con Stream API

```
```java
public static List<Proiezione> cercaProiezione(
    String titolo, String daGiorno, String daMese, String daAnno,
    String aGiorno, String aMese, String aAnno,
    String prezzoStr, String genere) {

    LocalDate daData = null;
    LocalDate aData = null;

    // Parsing date con gestione eccezioni
    try {
        if (daAnno != null && daMese != null && daGiorno != null) {
            daData = LocalDate.of(
                Integer.parseInt(daAnno),
                Integer.parseInt(daMese),
                Integer.parseInt(daGiorno)
            );
        }
        if (aAnno != null && aMese != null && aGiorno != null) {
            aData = LocalDate.of(
                Integer.parseInt(aAnno),
                Integer.parseInt(aMese),
                Integer.parseInt(aGiorno)
            );
        }
    } catch (DateTimeException | NumberFormatException e) {
        return new ArrayList<>();
    }

    double limitePrezzo = Double.MAX_VALUE;
    if (prezzoStr != null && !prezzoStr.isEmpty()) {
        try {
            limitePrezzo = Double.parseDouble(prezzoStr.replace(",", "."));
        } catch (NumberFormatException e) {
            System.err.println("Formato prezzo non valido");
        }
    }

    final LocalDate finalDaData = daData;
    final LocalDate finalAData = aData;
    final double finalLimitePrezzo = limitePrezzo;
```

```

return listaProiezioni.stream()
    .filter(p -> {
        // Filtro titolo (case-insensitive, substring match)
        if (titolo == null || titolo.trim().isEmpty()) return true;
        if (p.getTitolo() == null) return false;
        return p.getTitolo().toLowerCase()
            .contains(titolo.trim().toLowerCase());
    })
    .filter(p -> {
        // Filtro genere (exact match case-insensitive)
        if (genere == null || genere.isEmpty()) return true;
        if (p.getGenere() == null) return false;
        return p.getGenere().equalsIgnoreCase(genere.trim());
    })
    .filter(p -> p.getPrezzo() <= finalLimitePrezzo)
    .filter(p -> {
        // Filtro intervallo date
        LocalDate pData = p.getData();
        if (pData == null) return false;

        if (finalDaData != null && finalAData == null) {
            // Solo data inizio: exact match
            return pData.isEqual(finalDaData);
        } else if (finalDaData != null && finalAData != null) {
            // Intervallo completo
            return !pData.isBefore(finalDaData) &&
                !pData.isAfter(finalAData);
        } else if (finalDaData == null && finalAData != null) {
            // Solo data fine
            return !pData.isAfter(finalAData);
        } else {
            // Nessun filtro data
            return true;
        }
    })
    .collect(Collectors.toList());
}
...

```

Complessità algoritmica:

- Tempo: $O(n)$ dove n è il numero di proiezioni
- Spazio: $O(m)$ dove m è il numero di risultati

Vantaggi Stream API:

- Codice dichiarativo e leggibile
- Lazy evaluation per performance
- Facile parallelizzazione futura (`.parallel()`)

6.2 GestoreUtenti

Registrazione con Validazione

```
```java
public static void registraUtente(
 String nome, String cognome, String username,
 String password, String confermaPassword,
 String dataDiNascita, String domicilio)
 throws IllegalArgumentException {

 // Validazione campi obbligatori
 if (nome == null || cognome == null || username == null ||
 password == null || dataDiNascita == null) {
 throw new IllegalArgumentException(
 "Tutti i campi obbligatori devono essere compilati");
 }

 // Validazione corrispondenza password
 if (!password.equals(confermaPassword)) {
 throw new IllegalArgumentException(
 "Le password inserite non corrispondono");
 }

 // Verifica unicità username
 for (Utente u : listaUtenti) {
 if (u.getUsername().equals(username)) {
 throw new IllegalArgumentException(
 "Username già esistente");
 }
 }

 // Gestione campo opzionale
 String domicilioEffettivo = (domicilio == null) ? "N/D" : domicilio;

 // Creazione utente e cifratura password
 listaUtenti.add(new ClientiRegistrati(
 nome, cognome, username,
 Cifratura.cifra(confermaPassword),
 dataDiNascita, domicilioEffettivo
));

 salvaUtenti();
}
```
```

Login con Hashing

```

```java
public static Utente login(String username, String password) {
 String passwordCifrata = Cifratura.cifra(password);

 for (Utente u : listaUtenti) {
 if (u.getUsername().equals(username) &&
 u.getPassword().equals(passwordCifrata)) {
 return u;
 }
 }

 return null; // Credenziali errate
}
```

```

Complessità: $O(n)$ nel caso peggiore, dove n è il numero di utenti.

6.3 Sistema di Cifratura

Implementazione SHA-256

```

```java
public class Cifratura {
 public static String cifra(String password) {
 try {
 MessageDigest md = MessageDigest.getInstance("SHA-256");

 // Digest della password
 byte[] array = md.digest(password.getBytes());

 // Conversione a stringa esadecimale
 StringBuilder sb = new StringBuilder();
 for (byte b : array) {
 sb.append(String.format("%02x", b));
 }

 return sb.toString();

 } catch (Exception e) {
 System.err.println("Errore durante la cifratura: " +
 e.getMessage());
 return null;
 }
 }
}
```

```

****Caratteristiche SHA-256**:**

- Output: 256 bit (32 byte) = 64 caratteri esadecimali
- Irreversibile: Impossibile ottenere password da hash
- Deterministico: Stesso input → stesso hash
- Collision-resistant: Difficile trovare due input con stesso hash

7. Interfaccia Testuale (TUI)

7.1 Rendering con ANSI Escape Sequences

Controllo Cursore

```

```java
// Posizionamento cursore
System.out.print("\33[" + riga + ";" + colonna + "H");

// Pulizia schermo
System.out.print("\33[H33[2J");
System.out.flush();

// Ridimensionamento finestra terminale
System.out.print("\33[8;45;135t"); // 45 righe × 135 colonne
```

```

Box Drawing con Unicode

```
``java
private static String bordoSuperiore = "┌".repeat(130) + "┐";
private static String rigaVuota = "│" + " ".repeat(130) + "│";
private static String bordoMezzo = "├".repeat(130) + "┤";
private static String bordoInferiore = "└".repeat(130) + "┘";
``
```

****Caratteri Unicode Box-Drawing** (U+2500 – U+257F):**

- : Angoli doppi
- : Linee doppie orizzontali/verticali
- : Connettori

Logo ASCII Art

```
""java
static String[] logoCinemax = {
    "          [C] [I] [N] [E] [M] [A] [X]",
    "[C] [I] [N] [E] [M] [A] [X]",
    "[C] [I] [N] [E] [M] [A] [X]",
    "[C] [I] [N] [E] [M] [A] [X]",
    "[C] [I] [N] [E] [M] [A] [X]",
    "[C] [I] [N] [E] [M] [A] [X]"
};

public static void formattaLogo(String[] righelogo) {
    int lunghezzaRigaLogo = righelogo[1].length();
    int padding = (LARGHEZZA_MENU - lunghezzaRigaLogo) / 2;
    String pad = " ".repeat(padding);
```

```

        for (String linea : righelogo) {
            System.out.println("||" + pad +
                ((LARGHEZZA_MENU - lunghezzaRigaLogo) % 2 != 0 ? " " : "") +
                linea + pad + "||");
        }
    }
}
...

```

7.2 Gestione Input Dinamico

Form con Posizionamento ANSI

```

```java
public static boolean prendiDatiForm(String[] campiForm,
 String[] opzioniStatoMenu) {
 int altezzaLineaRichiesta = 10;
 int altezzaColonnaRichiesta = 52;

 for (int indiceOpzioni = 0, indiceRisultati = 0;
 indiceOpzioni < opzioniStatoMenu.length;
 indiceOpzioni++) {

 if (opzioniStatoMenu[indiceOpzioni].toUpperCase().equals("DATAINIZIO") ||
 opzioniStatoMenu[indiceOpzioni].toUpperCase().equals("DATAFINE")) {

 // Form data triplo (giorno, mese, anno)
 for (int i = 0; i < 3; i++) {
 System.out.print("33[" + altezzaLineaRichiesta + ";" +
 altezzaColonnaRichiesta + "H");
 String campoTmp = input.nextLine();
 String ris = convalidaInput(campoTmp);

 if (ris != null && ris.equals(":q"))
 return false;

 campiForm[indiceRisultati++] = ris;
 altezzaColonnaRichiesta += 12;
 }

 altezzaColonnaRichiesta = 52;
 altezzaLineaRichiesta += 3;

 } else {
 // Campo singolo
 System.out.print("33[" + altezzaLineaRichiesta + ";" +
 altezzaColonnaRichiesta + "H");
 String campoTmp = input.nextLine();

```

```

 String ris = convalidaInput(campoTmp);

 if (ris != null && ris.equals(":q"))
 return false;

 campiForm[indiceRisultati++] = ris;
 altezzaLineaRichiesta += 3;
 }
}

return true;
}
...

```

Comando speciale `:q`: Permette all'utente di annullare il form e tornare indietro. Implementa il pattern Command in modo semplificato.

Generazione Box Input

```

...java
public static String[] generaBoxInputUnicode(String opzione) {
 String[] boxInput = new String[3];

 String lineaSuperioreDinamica =
 "─".repeat(LARGHEZZA_BOX_INPUT - 6 - opzione.length());

 boxInput[0] = "┌─" + opzione.toUpperCase() + "─" +
 lineaSuperioreDinamica + "┐";
 boxInput[1] = "│> │";
 boxInput[2] = "└──────────────────────────────────┘";

 return boxInput;
}
...

```

### 7.3 Sistema di Paginazione

```

...java
public static void stampaPaginaProiezione() {
 List<Proiezione> listaPagina = FilmController.filmPaginaTmp;
 int paginaCorrente = FilmController.paginaCorrente;
 int elementiPagina = FilmController.ELEMENTI_PAGINA;

 if (listaPagina.isEmpty()) {
 formattaTesto("NESSUNA PROIEZIONE TROVATA", "centro", true);
 formattaTesto("PREMI INVIO PER TORNARE INDIETRO", "centro", true);
 return;
 }
}

```

```

// Info pagina
int totalePagine = (FilmController.proiezioniTrovate.size() +
 elementiPagina - 1) / elementiPagina;
String infoPagina = "PAGINA " + (paginaCorrente + 1) +
 " DI " + totalePagine;
formattaTesto(infoPagina, "sinistra", true);

System.out.println(bordoMezzo);
System.out.println(rigaVuota);

// Visualizzazione proiezioni
int indiceSceltaMenu = 1;
for (Proiezione proiezione : listaPagina) {
 String rigaFilm = "[" + indiceSceltaMenu++ + "] " +
 proiezione.toString();
 formattaTesto(rigaFilm, "sinistra", true);
}

System.out.println(rigaVuota);
System.out.println(bordoMezzo);
System.out.println(rigaVuota);

// Controlli navigazione
if (FilmController.esistenzaPaginaSuccessiva)
 formattaTesto("[N] PAGINA SUCCESSIVA", "centro", true);
if (paginaCorrente > 0)
 formattaTesto("[B] PAGINA PRECEDENTE", "centro", true);
formattaTesto("[C] ANNULLA / ESCI", "centro", true);
}
...

```

## 8. Scelte Algoritmiche

### 8.1 Algoritmo di Ricerca con Stream API

Complessità Temporale

...

Operazione	Complessità
----- -----	
Filtro titolo (substring)	$O(n \times m)$ // n proiezioni, m lunghezza titolo
Filtro genere (equals)	$O(n)$
Filtro prezzo	$O(n)$
Filtro date	$O(n)$
Totale (sequenziale)	$O(n \times m)$

...

Ottimizzazione Possibile

Per dataset molto grandi (>10,000 proiezioni), si potrebbe implementare:

1. Indici: HashMap per titoli e generi
2. B-Tree: Per range query su date e prezzi
3. Parallel Stream: `listaProiezioni.parallelStream()`

### 8.2 Gestione Paginazione

Algoritmo

```
```java
int indiceInizio = ELEMENTI_PAGINA * paginaCorrente;
int indiceFine = Math.min(
    indiceInizio + ELEMENTI_PAGINA,
    proiezioniTrovate.size()
);

filmPaginaTmp = proiezioniTrovate.subList(indiceInizio, indiceFine);
```
```

Complessità:

- Spazio:  $O(k)$  dove  $k = ELEMENTI\_PAGINA$
- Tempo:  $O(1)$  con `subList()` (vista, non copia)

### 8.3 Validazione Input

Pattern Null-Object

```
```java
public static String convalidaInput(String campoTmp) {
    if (campoTmp.equals(":q")) {
```

```
        CineMax.stackRecord.pop();  
        return ":q";  
    }  
  
    if (campoTmp.equals("")) {  
        return null; // Campo opzionale  
    }  
  
    return campoTmp;  
}  
...
```

Uso di `null` per rappresentare campi opzionali non compilati, semplificando la logica di validazione nei gestori.

9. Gestione Eccezioni

Gerarchia Eccezioni Gestite

```
...  
Exception  
├── IllegalArgumentException → Validazione input  
├── NumberFormatException → Parsing numeri  
├── DateTimeException → Parsing date  
├── ArrayIndexOutOfBoundsException → Accesso array  
└── IOException → File I/O  
...
```

Pattern Try-Catch Multi-Livello

```
```java  
// Livello Controller
try {
 // Logica business
} catch (SpecificException e) {
 messaggioErroreCorrente = "Messaggio specifico";
 stackRecord.push(STATO_ERRORE);
} catch (Exception e) {
 messaggioErroreCorrente = "Errore generico";
 stackRecord.push(STATO_ERRORE);
}

// Livello DAO
try {
 // Operazione file
} catch (IOException e) {
 System.err.println("Errore I/O: " + e.getMessage());
}
```
```

Stato Errore Dedicato

```
```java  
STATO_ERRORE(
 new String[]{"premi invio per tornare indietro"},
 true, "error", "centro"
) {
 @Override
 public void eseguiLogicaAssociata() {
 input.nextLine();
 CineMax.stackRecord.pop(); // Torna automaticamente indietro
 }
}
```
```


10. JavaDoc e Documentazione Codice

Struttura JavaDoc

Per generare la documentazione JavaDoc completa del progetto, seguire questi passaggi:

Generazione JavaDoc

```
```bash
Dalla cartella root del progetto
javadoc -d doc -sourcepath src -subpackages cinemax \
 -encoding UTF-8 -charset UTF-8 -docencoding UTF-8 \
 -windowtitle "CineMax - Documentazione API" \
 -doctitle "CineMax - Sistema Gestione Cinematografica" \
 -header "Laboratorio A - 2026" \
 -author -version -private
...

```

#### **\*\*Parametri\*\*:**

- `-d doc`: Output nella cartella `doc`
- `-subpackages cinemax`: Include tutti i package
- `-private`: Includi anche membri privati
- `-author -version`: Mostra tag `@author` e `@version`

#### **##### Tag JavaDoc Utilizzati**

```
```java
/**
 * Descrizione breve della classe.
 * <p>
 * Descrizione dettagliata con più paragrafi.
 * Può includere esempi di utilizzo.
 *
 * @author Modena Matteo (Matricola: 765099) - VA
 * @author Baroncelli Luca (Matricola: 761582) - VA
 * @author Bin Alessio (Matricola: 762387) - VA
 * @version 1.0
 * @since 1.0
 */
public class EsempioClasse {

    /**
     * Descrizione del metodo.
     * <p>
     * Descrizione dettagliata del comportamento.
     *
     * @param username Username dell'utente
    }
}

```

```

* @param password Password in chiaro (verrà cifrata)
* @return Oggetto {@link Utente} se credenziali valide, <code>null</code> altrimenti
* @throws IllegalArgumentException se username o password sono null
* @see Cifratura#cifra(String)
*/
public static Utente login(String username, String password)
    throws IllegalArgumentException {
    // Implementazione
}
}
...

```

Organizzazione Package

```

...
doc/
├── index.html          # Indice principale
├── allclasses-index.html # Tutte le classi
├── overview-tree.html  # Gerarchia classi
├── cinemax/
│   ├── package-summary.html
│   ├── CineMax.html
│   ├── MenuMangaer.html
│   ├── controller/
│   │   ├── package-summary.html
│   │   ├── FilmController.html
│   │   └── AutenticazioniController.html
│   ├── model/
│   │   ├── package-summary.html
│   │   ├── Utente.html
│   │   └── Proiezione.html
│   └── util/
│       ├── package-summary.html
│       ├── GestoreUtenti.html
│       └── GestoreProiezione.html
└── resources/
...

```

File package-info.java (Consigliato)

Creare file `package-info.java` in ogni package per documentazione a livello di package:

```

```java
/**
 * Package controller contiene le classi che gestiscono la logica
 * di business dell'applicazione, fungendo da intermediari tra
 * la View (TUI) e il Model/DAO.
 * <p>

```

```
* Classi principali:
*
* {@link cinemax.controller.FilmController} - Gestione proiezioni
* {@link cinemax.controller.AutenticazioniController} - Autenticazione
*
*
* @since 1.0
* @version 1.0
*/
package cinemax.controller;
...
```

---

## 11. Considerazioni su Performance e Scalabilità

### Analisi Performance Attuale

#### Operazioni I/O

...

Operazione	Frequenza	Complessità
Carica utenti	1× all'avvio	O(n)
Carica proiezioni	Per ricerca	O(m)
Salva utenti	Per registrazione	O(n)
Salva proiezioni	Per inserimento	O(m)

...

Bottleneck: Caricamento completo file CSV ad ogni ricerca.

#### Memoria

```
```java
// Stima utilizzo memoria (worst case)
List<Proiezione> listaProiezioni; // ~1KB per proiezione
// 10,000 proiezioni = ~10MB

List<Utente> listaUtenti; // ~500B per utente
// 1,000 utenti = ~500KB

Stack<StatoMenu> stackRecord; // Trascurabile (<1KB)
```
```

#### Ottimizzazioni Possibili

##### 1. Lazy Loading

```
```java
// Attuale: carica tutto
public static void leggiProiezioni() {
    listaProiezioni.clear();
    // Carica tutte le proiezioni
}

// Ottimizzato: carica su richiesta
public static void leggiProiezioniRange(LocalDate da, LocalDate a) {
    // Carica solo proiezioni nell'intervallo
}
```
```

##### 2. Caching con Timestamp

```

```java
private static long lastLoadTime = 0;
private static final long CACHE_VALIDITY = 300_000; // 5 minuti

public static void leggiProiezioniCached() {
    long now = System.currentTimeMillis();
    if (now - lastLoadTime > CACHE_VALIDITY) {
        leggiProiezioni();
        lastLoadTime = now;
    }
}
```

```

### 3. Indici per Ricerca

```

```java
// Indice per titolo
private static Map<String, List<Proiezione>> indiceTitoli = new HashMap<>();

// Indice per genere
private static Map<String, List<Proiezione>> indiceGeneri = new HashMap<>();

// Costruzione indici
public static void costruisciIndici() {
    for (Proiezione p : listaProiezioni) {
        indiceTitoli
            .computeIfAbsent(p.getTitolo().toLowerCase(), k -> new ArrayList<>())
            .add(p);
        indiceGeneri
            .computeIfAbsent(p.getGenere(), k -> new ArrayList<>())
            .add(p);
    }
}
```

```

**\*\*Risultato\*\*:** Ricerca  $O(1)$  invece di  $O(n)$  per titolo esatto.

### 4. Database Relazionale

Per scalabilità oltre 10,000 proiezioni, considerare migrazione a:

```

```sql
-- Schema SQLite/PostgreSQL
CREATE TABLE proiezioni (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    data DATE NOT NULL,
    ora TIME NOT NULL,

```

```
titolo VARCHAR(255) NOT NULL,  
genere VARCHAR(50),  
regista VARCHAR(100),  
anno INTEGER,  
durata INTEGER,  
eta_minima INTEGER,  
prezzo DECIMAL(5,2),  
posti_sala INTEGER,  
UNIQUE (titolo, data, ora)  
);  
  
CREATE INDEX idx_data ON proiezioni(data);  
CREATE INDEX idx_titolo ON proiezioni(titolo);  
CREATE INDEX idx_genere ON proiezioni(genere);  
'''
```

12. Bibliografia Tecnica

Documentazione Tecnica

1. [Oracle Java Documentation](#)

- Consultato per: API Java, gestione I/O, strutture dati

2. [Java Tutorials - Oracle](#)

- Consultato per: Best practices, pattern di programmazione

Standard e Specifiche

1. [ANSI Escape Codes](#)

- Consultato per: Sequenze di escape per controllo cursore e formattazione testuale

2. [Unicode Standard](#)

- Consultato per: Caratteri box-drawing e simboli grafici

Materiale Didattico

1. Documentazione dei Corsi

Note Finali

Per assistenza tecnica o segnalazione bug, contattare gli sviluppatori:

- Bin Alessio - abin@studenti.uninsubria.it
- Modena Matteo - mmodena@studenti.uninsubria.it
- Baroncelli Luca - lbaroncelli@studenti.uninsubria.it